# CIS*3110 Operating Systems

## Week 6

## Lecture #10 Virtualization: Page faults, swapping

**Prof. Yulia Kotseruba**

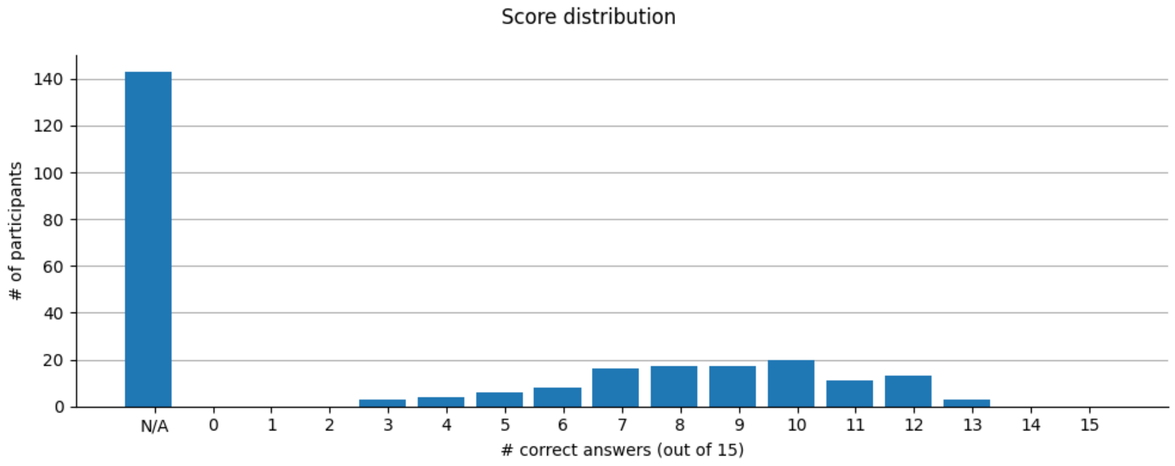
**February 10, 2026**

# Quiz 8

- **57%** average (down from 61% in quiz 7)
- 3 hardest questions:
  - (19%) What does this code print to stdout? a) Hi! Bye! b) Hi! c) nothing d) will not compile
  - (39%) How many times will "A" be printed? a) 0 b) 1 c) 3 d) 4
  - (43%) What is the output of this code? a) will not compile b) undefined behavior c) address of ptr  d) hi?

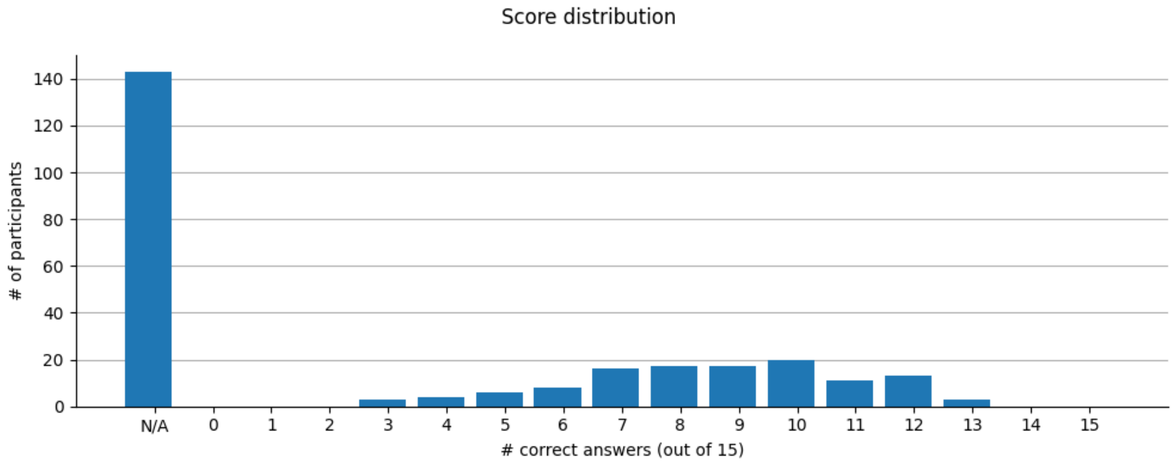
```c
int main() {
    char s1[] = "Hi?", *s2 = "Bye?";
    s1[2] = '!';
    printf("%s\n", s1);
    s2[3] = '!';
    printf("%s\n", s2);
}
```

```c
int main() {
    if (fork() && fork()) {
        printf("A\n");
    }
}
```



# Quiz 8

- **57%** average (down from 61% in quiz 7)
- 3 hardest questions:
  - (19%) What is the output of this code? a) Hi! Bye! b) Hi! c) nothing d) will not compile
  - (39%) How many times will "A" be printed? a) 0 b) 1 c) 3 d) 4
  - (43%) What is the output of this code? a) will not compile b) undefined behavior c) address of ptr  d) hi?




```c
char* get_c() {
    char s[] = "hi!"; return s;
}
int main() {
    char *c = get_c();
    c[2] = '?';
    printf("%s\n", c);
}
```

# Recap

- **Paging** splits memory into **equally-sized** chunks — simpler and more efficient memory allocation
  - Pros
    - fixed-size pages avoid **external fragmentation**
    - **easy to allocate** memory (all blocks are the same, any free one will do)
    - does **not** require hardware support - flexible
  - Cons
    - does not eliminate **internal fragmentation** (if page size is too big)
    - page tables are **large**
    - accessing memory is **slow** (**doubles** the number of memory operations per instruction) -> TLB helps with this

# Recap

- **Translation Lookaside Buffer (TLB)** - memory **cache** for storing translations of virtual memory to physical memory addresses
  - Pros
    - **reduces** memory accesses to page table
    - **very fast** hardware-supported implementation
  - Cons
    - needs **careful management** to avoid too many **TLB misses**

# Recap

- Problem: How to **reduce memory** required for storing **page tables**?
- Option #1: **larger pages** - reduce page table size, but lead to internal fragmentation
- Option #2: **segmentation for page tables** - no internal fragmentation, but issues with complexity
- Option #3: **multi-level page tables** - turn page table into a tree-like structure
  - less internal fragmentation, but high complexity and slow memory access
- Option #4: **inverted page tables** - index physical pages - **one table for all processes**
  - saves memory, but issues with search and processes sharing memory

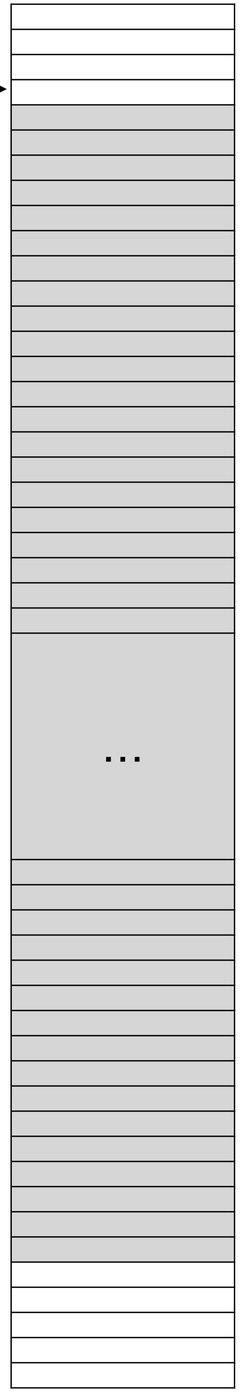
# Swapping

- Address space of processes is larger than RAM
- This is the point of virtualization - illusion of infinite space
- **Swapping** helps maintain this illusion
  - Use hard drive to extend the size of RAM

# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

processes access memory at virtual address



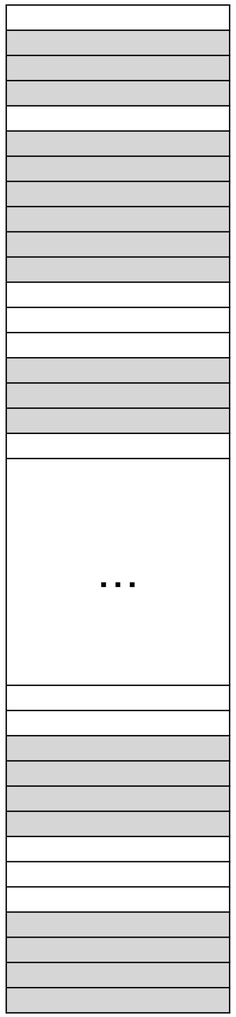

virtual address space

each process has its own

split into pages of equal size

equal size of pages and frames avoids external fragmentation

physical memory is divided into frames with same size as pages



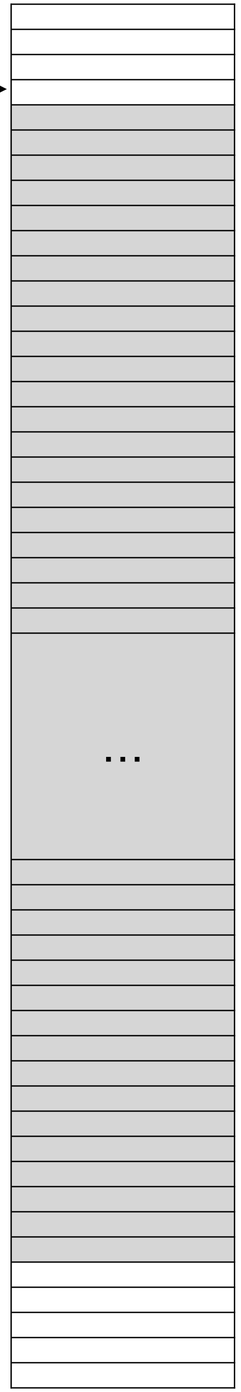

RAM

# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

processes access memory at virtual address

virtual address space

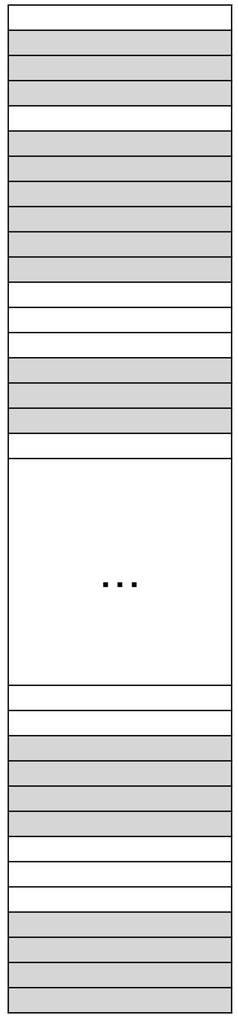



pages of virtual memory can be anywhere in physical memory

to access value of the variable need to translate virtual address to physical address

need to keep track of mapping between virtual pages and physical frames

offset is the same because page size and frame size are the same



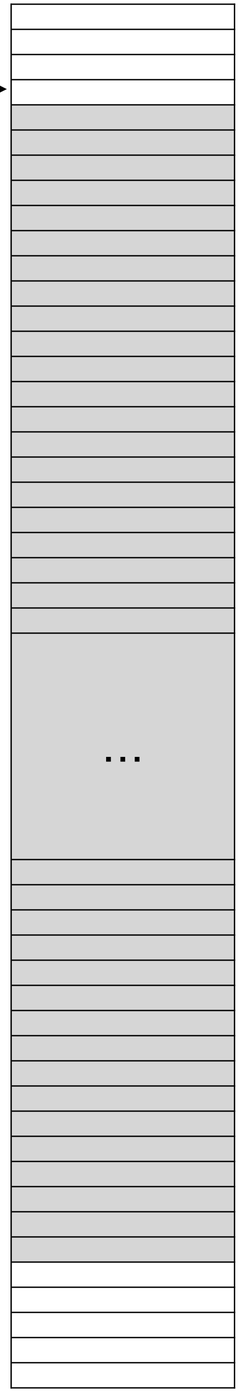

RAM

# Paging

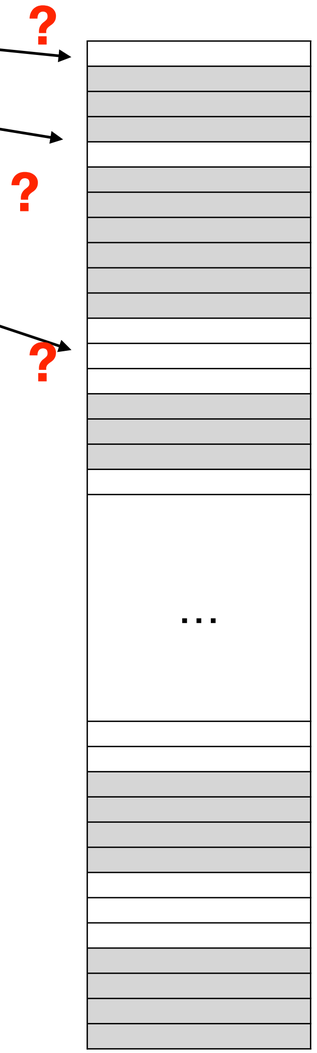
```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

processes access memory at virtual address

virtual address space



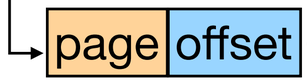

where is this page in RAM?



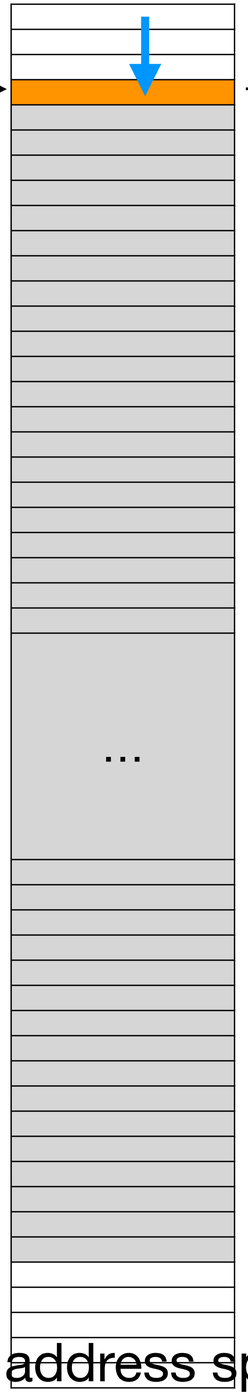

RAM

# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```



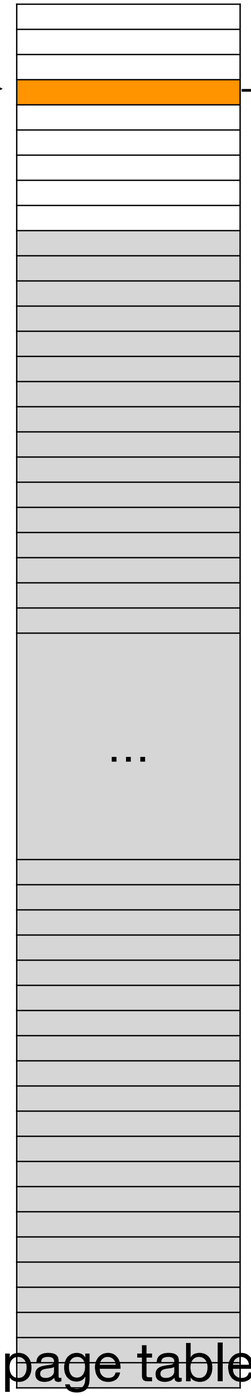

processes access memory at virtual address



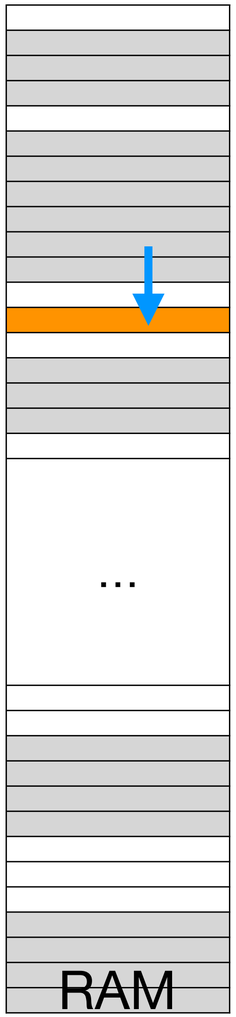

virtual address space

indexed by virtual pages

each entry contains a corresponding frame

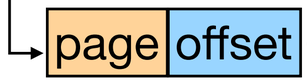



page table

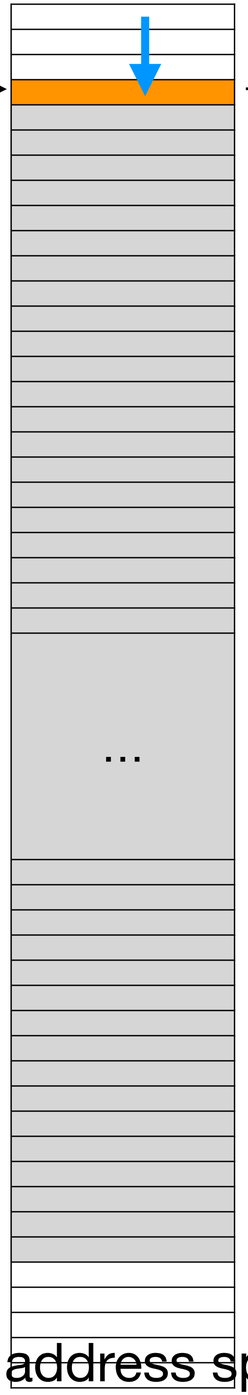


# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```




processes access memory at virtual address

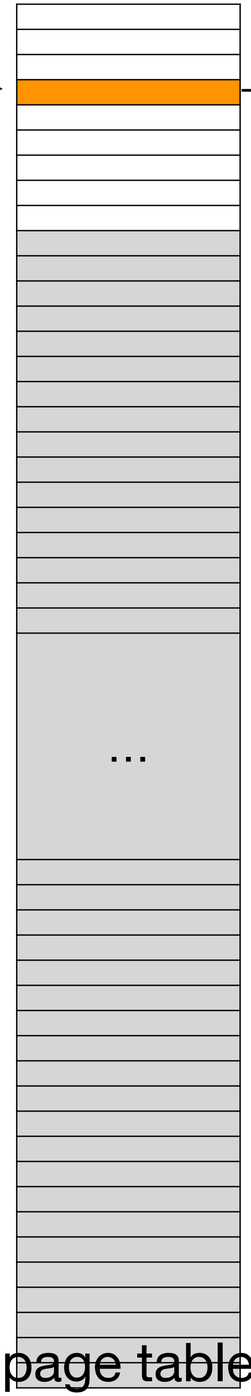



virtual address space

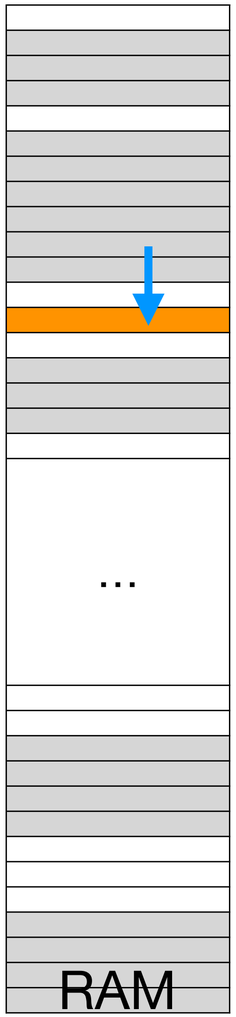
problem #1: page tables are large

indexed by virtual pages

each entry contains a corresponding frame




page table

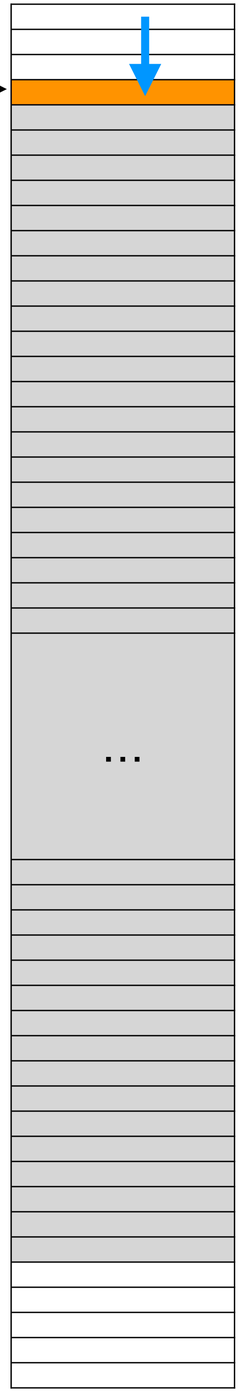


# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

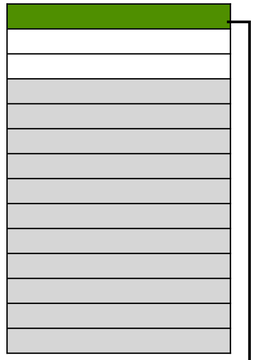
processes access memory at virtual address

virtual address space

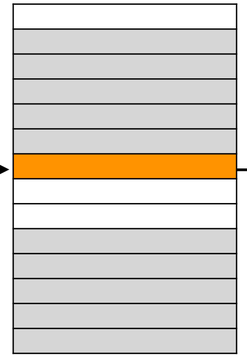



problem #1: page tables are large

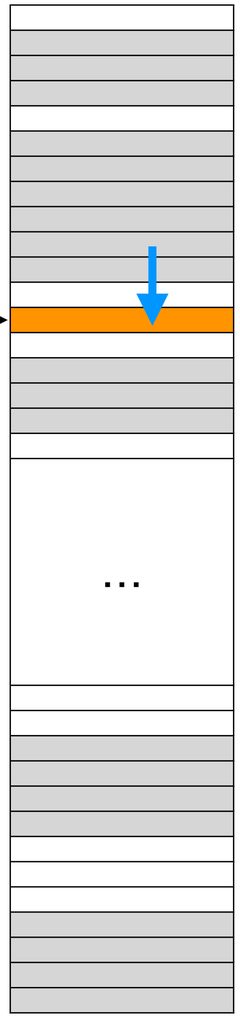
solution: multi-level page tables



page table directory



page table



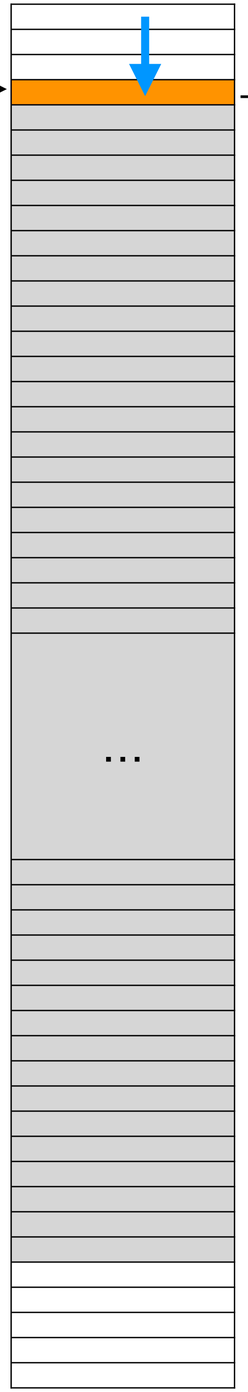

RAM

# Paging

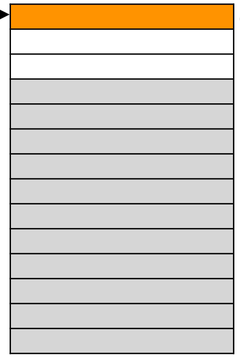
```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

processes access memory at virtual address

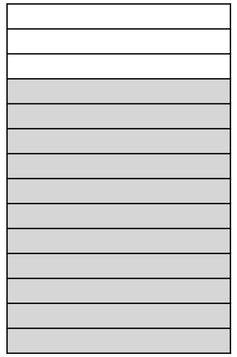
virtual address space



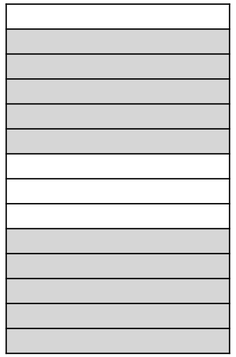

TLB hit



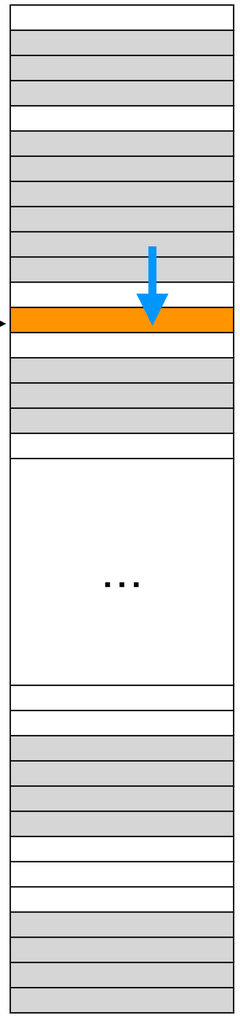
TLB



page table directory



page table



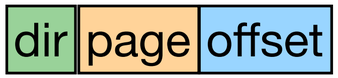

RAM

problem #2: too many memory accesses during addressing

Solution: a small buffer to cache address translations

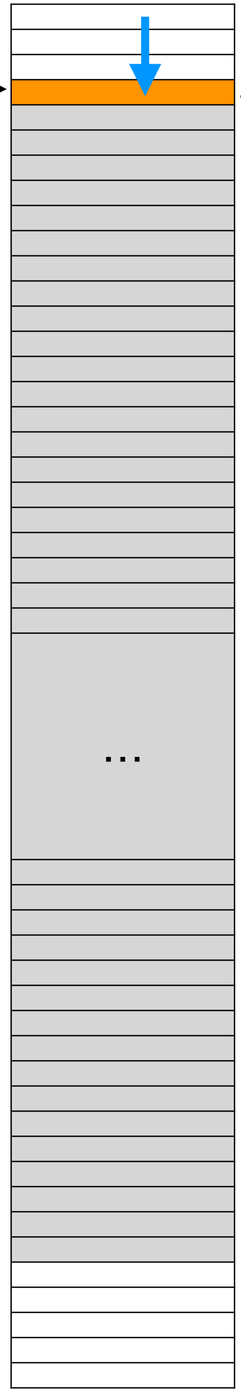
# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

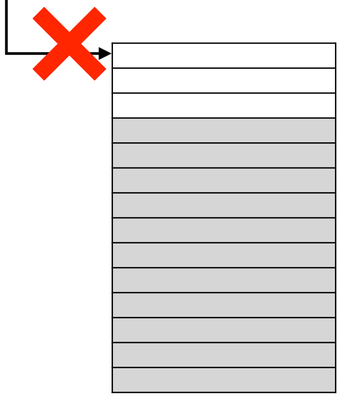



processes access memory at virtual address

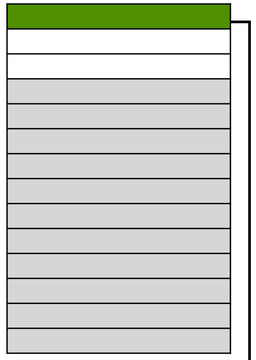
virtual address space




TLB miss

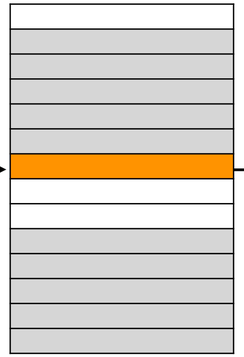


TLB

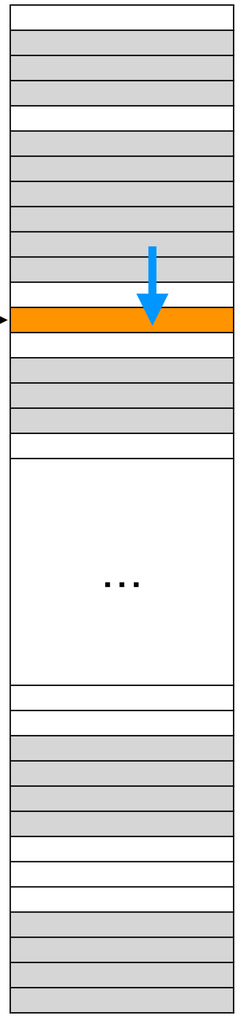


page table directory

go through page table and update TLB



page table



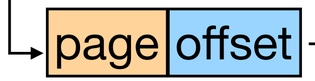

RAM

problem #2: too many memory accesses during addressing

Solution: a small buffer to cache address translations

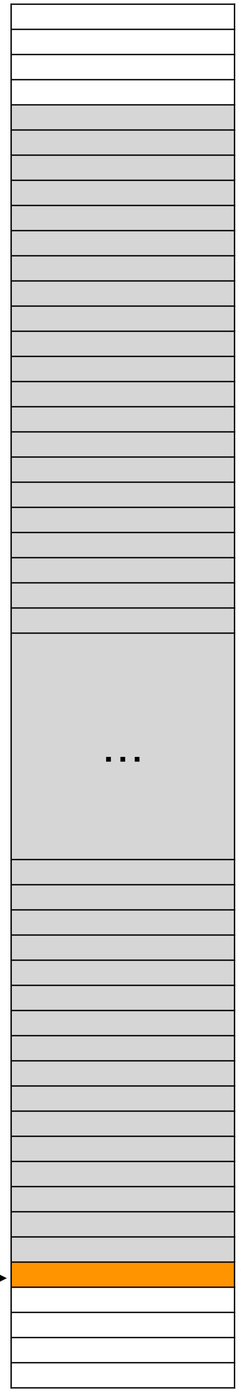
# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```




processes access memory at virtual address

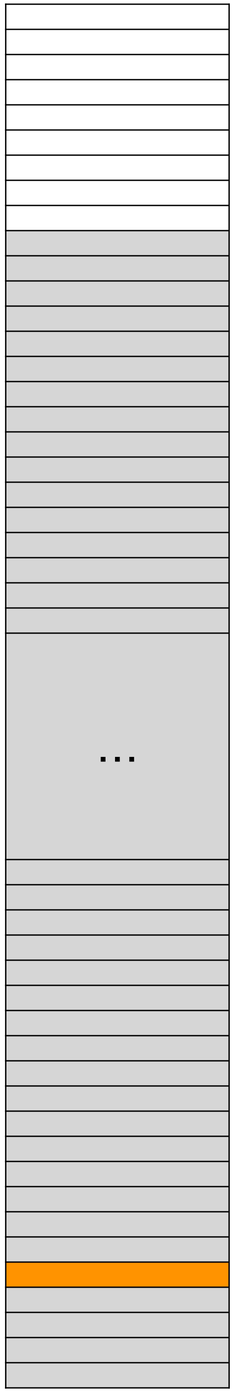
virtual address space




what happens when the page table does not have a frame for our page?

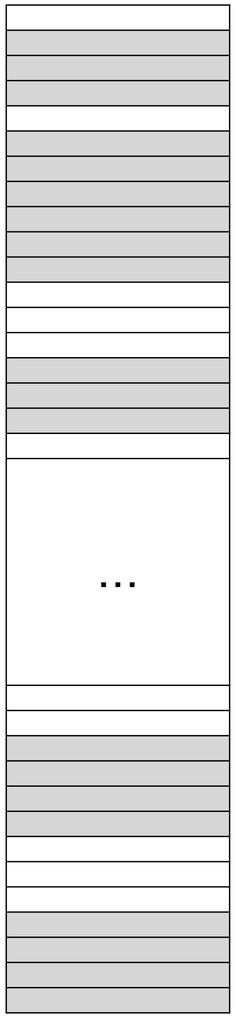
page fault

page table




the frame corresponding to the page is not in RAM

?



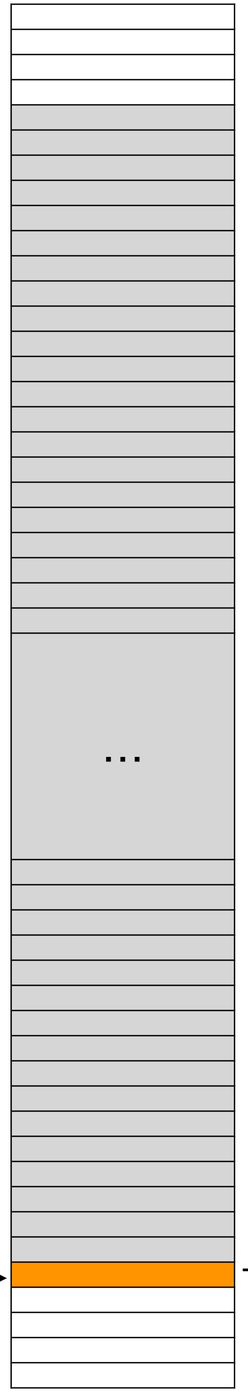

RAM

# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```

processes access memory at virtual address

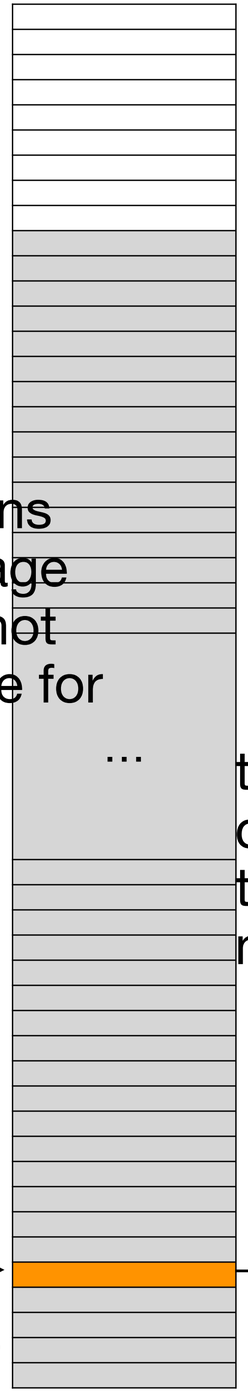
virtual address space




what happens when the page table does not have a frame for our page?

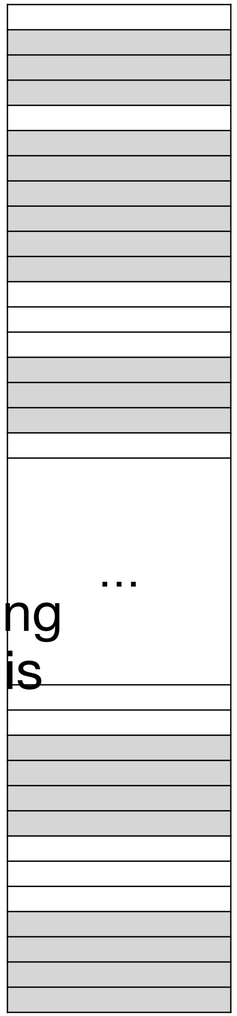
page fault

page table



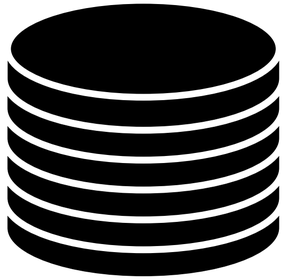

the frame corresponding to the page is not in RAM

?




RAM

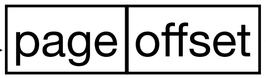
the frame is on the hard drive



HD (swap space)

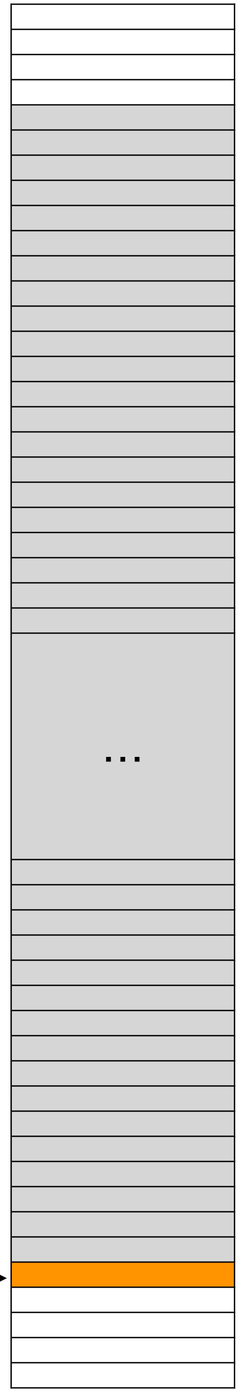
# Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```




processes access memory at virtual address

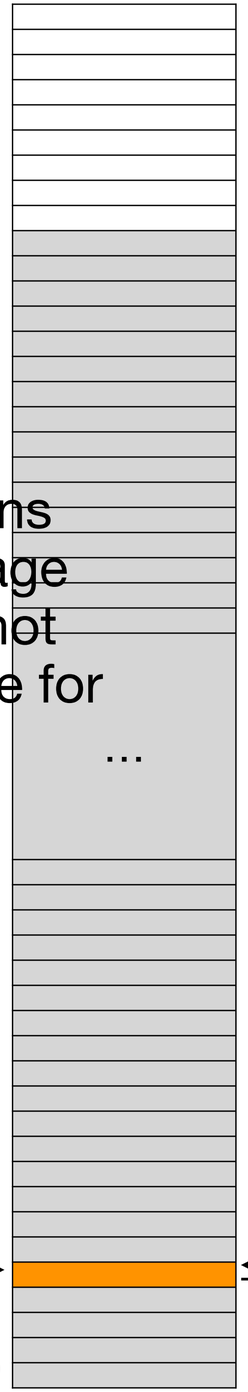
virtual address space



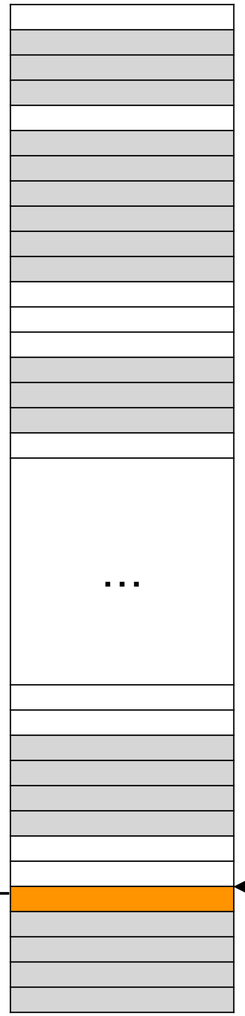

what happens when the page table does not have a frame for our page?

page fault

page table

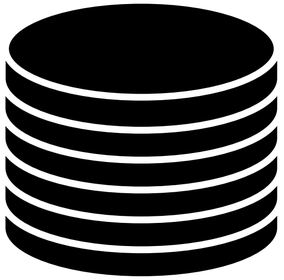



update page table




RAM

load the page into RAM



HD (swap space)

need to get the page from HD

# Page table organization

- Assume a linear page table - an array
- Array index is the virtual page number (VPN)

| | |
|---|---|
| 0 | frame 3 |
| 1 | ... |
| ... | ... |
| x | frame Y |

- Page-table entry (PTE) contains
  - physical frame number (PFN)
  - **valid bit** - marks invalid addresses in the virtual address space (e.g. free space between stack and heap)
  - **present bit** - if this page is in the physical memory
  - **dirty bit** - has this page been modified
  - **reference bit** (aka accessed bit) - has this page been accessed
  - **permissions** - is this address read-only or writable

# Page fault

TLB hit

```
VPN = (virtual_address & VPN_MASK) >> SHIFT
(success, TlbEntry) = TLB_Lookup(VPN)
if (success == True)
    if (CanAccess(TlbEntry.ProtectBits) == True)
      //access OK
      offset = virtual_address & OFFSET_MASK
      phys_addr = (PTE.PFN << PFN_SHIFT) | offset
      register = AccessMemory(phys_addr)
    else
      RaiseException(PROTECTION_FAULT)
```

get virtual page number

check if it's in the TLB

check if it is not protected

get the offset

combine it with frame number from the cached page table entry (PTE)

otherwise raise memory fault

# Page fault

TLB miss

```
// continued from previous slide
else //TLB miss
    PTEAddr = PTBR + (VPN * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)
    if (PTE.valid == False)
      RaiseException(SEGMENTATION_FAULT)
    else
      if (CanAccess(PTE.ProtectBits) == False)
        RaiseException(PROTECTION_FAULT)
      else
        if (PTE.Present == True)
          TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
          RetryInstruction()
        else if (PTE.Present == False)
          RaiseException(PAGE_FAULT)
```

get the address of the page table

get the page table entry (PTE)

check if page is valid

check if page is not protected

check if page is present

update the TLB

otherwise, page is not in RAM

we've got ourselves a page fault

# Page fault

- Page fault* - page is not present in physical memory
  - its present bit in page table entry (PTE) is set to 0
- The missing page is on the hard drive
  - Need to fetch it, load it to RAM, update the page table entry

* could have been called a miss (like a TLB miss), but missing pages are harder to deal with, so they are called faults

# Page fault

```
PFN = FindFreePhysicalPage()
if (PFN == -1)
    PFN = EvictPage()
DiskRead(PTE.DiskAddr, PFN)
PTE.present = True
PTE.PFN = PFN
RetryInstruction()
```

find a free physical page in RAM

if there are none, evict some pages

load the page from hd
update page table

rerun the instruction that caused page fault

# Swapping

- RAM holds only a subset of pages that are used by all processes running in the system
  - RAM itself is a **cache** for virtual memory
- **Swap space** on hard drive is used to **extend** RAM
- Pages are **evicted** from RAM in batches
- RAM has a high-watermark (HW) and low-watermark (LW)
  - when there are fewer than LW pages available
  - swap daemon (aka page daemon) starts evicting pages from RAM
  - stops when there are HW pages available

# Cache management

- As with TLB before, RAM needs to be carefully managed
- Page faults are **expensive** - loading from hard drive is **orders of magnitude** slower than accessing RAM
- Page faults and swapping should be **avoided** as much as possible
- Goal: maximize average memory access time (AMAT)

$$\text{AMAT} = T_M + (P_{miss} \cdot T_D)$$

cost of accessing RAM

probability of miss $\in [0,1]$

cost of accessing hard drive

there is **always** a cost of accessing RAM, but when there is a miss, there is an **additional cost** of accessing the hard drive

$P_{miss}$ should be as **small** as possible

# AMAT example

| page | offset |
|---|---|
| 4 bits | 8 bits |

- address space: 4KB with 256-byte pages
- # pages: 4096/256 =16
- virtual address: 12 bits (2^12 = 4096) = 4 bits for page + 8 bits for offset
- assume all pages except page #3 are in RAM already
- a process generates the following virtual addresses

addresses of the first byte of each of the first 10 pages

| hex | 0x000 | 0x100 | 0x200 | 0x300 | 0x400 | 0x500 | 0x600 | 0x700 | 0x800 | 0x900 |
|---|---|---|---|---|---|---|---|---|---|---|
| dec | 0 | 256 | 512 | 768 | 1024 | 1280 | 1536 | 1792 | 2048 | 2304 |
| | hit | hit | hit | miss | hit | hit | hit | hit | hit | hit |

what is the hit rate? **90%** (9 out 10 pages are in memory)

what is the miss rate? **10%** ($P_{hit}$+$P_{miss}$=100%)

$T_M$=10ns, $T_D$=10ms

$$\text{AMAT} = T_M + (P_{miss} \cdot T_D) = 100ns + 0.1 * 10ms = 1.0001ms$$

what if hit rate is 99%?

$$\text{AMAT} = T_M + (P_{miss} \cdot T_D) = 100ns + 0.01 * 10ms = 0.011ms$$

100x improvement

# Page replacement policy

- Problem: cache is small. Which page should be removed?
  - Option #1: **MIN** - evict the page that will not be used in the future

# MIN

- MIN (aka optimal) - remove the page which will be accessed furthest in the future - fewest-possible cache misses
- Proposed by Les Belady in 1966 at IBM research
- MIN is also known as Belady's algorithm

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

| |
|---|
| 0 |
| 1 |
| 2 |

cache

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | | | |
| 0 | | | |
| 3 | | | |
| 1 | | | |
| 2 | | | |
| 1 | | | |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

cache is full, which page to evict?

MIN evicts the page that will be used further in the future

| |
|---|
| 0 |
| 1 |
| 2 |

cache

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | | | |
| 0 | | | |
| 3 | | | |
| 1 | | | |
| 2 | | | |
| 1 | | | |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

| |
|---|
| 0 |
| 1 |
| 2 |

cache

cache is full, which page to evict?

MIN evicts the page that will be used further in the future

0 and 1 will be used soon, 2 will be used the latest -> evict 2

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | | | |
| 0 | | | |
| 3 | | | |
| 1 | | | |
| 2 | | | |
| 1 | | | |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

cache is full, which page to evict?

MIN evicts the page that will be used further in the future

0 and 1 will be used soon, 2 will be used the latest -> evict 2

| |
|---|
| 0 |
| 1 |
| 3 |

cache

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 2 | 0,1,3 |
| 0 | | | |
| 3 | | | |
| 1 | | | |
| 2 | | | |
| 1 | | | |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

cache is full again which page to evict?

| |
|---|
| 0 |
| 1 |
| 3 |

cache

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 2 | 0,1,3 |
| 0 | Hit | | 0,1,3 |
| 3 | Hit | | 0,1,3 |
| 1 | Hit | | 0,1,3 |
| 2 | Miss | | |
| 1 | | | |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

cache is full again which page to evict?

1 will be accessed soon, can remove either 0 or 3

| |
|---|
| 0 |
| 1 |
| 2 |

cache

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 2 | 0,1,3 |
| 0 | Hit | | 0,1,3 |
| 3 | Hit | | 0,1,3 |
| 1 | Hit | | 0,1,3 |
| 2 | Miss | 3 | 0,1,2 |
| 1 | | | |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

cache is full again which page to evict?

1 will be accessed soon, can remove either 0 or 3

| |
|---|
| 0 |
| 1 |
| 2 |

cache

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 2 | 0,1,3 |
| 0 | Hit | | 0,1,3 |
| 3 | Hit | | 0,1,3 |
| 1 | Hit | | 0,1,3 |
| 2 | Miss | 3 | 0,1,2 |
| 1 | Hit | | 0,1,2 |

# MIN

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0 1 2 0 1 3 0 3 1 2 1

The first access to any page is compulsory

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 2 | 0,1,3 |
| 0 | Hit | | 0,1,3 |
| 3 | Hit | | 0,1,3 |
| 1 | Hit | | 0,1,3 |
| 2 | Miss | 3 | 0,1,2 |
| 1 | Hit | | 0,1,2 |

| |
|---|
| 0 |
| 1 |
| 2 |

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.5\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{6}{6+1} = 85.7\,\%$$

# MIN

- MIN (aka optimal) - remove the page which will be accessed furthest in the future - **fewest-possible cache misses**
- Problem - "Prediction is very difficult, especially if it’s about the future" (attributed to Niels Bohr)
- This is the **upper bound** on how good any policy can get
  - useful for comparisons, e.g. if you know that the best possible hit rate is 82%, then getting an 80% is not too bad

# Page replacement policy

- Problem: cache is small. Which page should be removed?
  - Option #1: **MIN** - evict the page that will not be used in the future
    - cons: need to predict the future
  - Option #2: **FIFO** - evict the **oldest** page

# FIFO

- First-in, first-out replacement
  - items are added to the queue in the order of arrival
  - item that has been in the cache the **longest** is the first to be evicted

| | | |
|---|---|---|
| A | B | C |

D

# FIFO

- First-in, first-out replacement
  - items are added to the queue in the order of arrival
  - item that has been in the cache the **longest** is the first to be evicted

| | | | |
|---|---|---|---|
| B | C | D | E |

# FIFO

- First-in, first-out replacement
  - items are added to the queue in the order of arrival
  - item that has been in the cache the **longest** is the first to be evicted

| | | |
|---|---|---|
| C | D | E |

# FIFO

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

| |
|---|
| 0 |
| 1 |
| 2 |

What is the hit rate?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{4}{4+7} = 36.3\,\%$$

What is the hit rate without the compulsory misses?

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{4}{4+3} = 57.1\,\%$$

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 0 | 1,2,3 |
| 0 | Miss | 1 | 2,3,0 |
| 3 | Hit | | 2,3,0 |
| 1 | Miss | 2 | 3,0,1 |
| 2 | Miss | 3 | 0,1,2 |
| 1 | Hit | 1 | 0,1,2 |

MIN was 54.5% and 85.7%

# Belady’s anomaly

- Belady and colleagues showed a strange behavior or FIFO replacement policy
- Assumption: when cache size increases, hit rate should improve
- But FIFO hit rate **decreased** with larger cache size

# Belady's anomaly

- Assume cache that fits **4** pages
- Program accesses 5 virtual pages in this order:
  - 0 1 2 3 0 1 4 0 1 2 3 4

cache size 3

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{3}{3+9} = 25\,\%$$

cache size 4

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{2}{2+10} = 17\,\%$$

a large cache does not guarantee high hit rate for all policies and all reference strings

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 3 | Miss | | 0,1,2,3 |
| 0 | Hit | | 0,1,2,3 |
| 1 | Hit | | 0,1,2,3 |
| 4 | Miss | 0 | 1,2,3,4 |
| 0 | Miss | 1 | 2,3,4,0 |
| 1 | Miss | 2 | 3,4,0,1 |
| 2 | Miss | 3 | 4,0,1,2 |
| 3 | Miss | 4 | 0,1,2,3 |
| 4 | Miss | 0 | 1,2,3,4 |

# Page replacement policy

- Problem: cache is small. Which page should be removed?
  - Option #1: **MIN** - evict the page that will not be used in the future
    - cons: need to predict the future
  - Option #2: **FIFO** - evict the oldest page
    - cons: more likely to throw out heavily used pages, Belady's anomaly
  - Option #3: **Random** - evict a page chosen at random

# Random

- Random policy - pick a random page to evict
- Simple to implement, not intelligent

# Random

- Assume cache that fits 3 pages
- Program accesses 4 virtual pages in this order:
  - 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

MIN $$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.5\,\%$$

FIFO $$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{4}{4+7} = 36.3\,\%$$

**Random** $$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{5}{5+7} = 42\,\%$$

| Access | Hit/ Miss? | Evict | Cache |
|---|---|---|---|
| 0 | Miss | | 0 |
| 1 | Miss | | 0,1 |
| 2 | Miss | | 0,1,2 |
| 0 | Hit | | 0,1,2 |
| 1 | Hit | | 0,1,2 |
| 3 | Miss | 0 | 1,2,3 |
| 0 | Miss | 1 | 2,3,0 |
| 3 | Hit | | 2,3,0 |
| 1 | Miss | 3 | 2,0,1 |
| 2 | Hit | | 2,0,1 |
| 1 | Hit | | 2,0,1 |

# Random

- Random policy - pick a random page to evict
- Simple to implement, not intelligent
- When tested **10K times** on the sequence form the previous slide with different random seeds
  - **~40%** of the time was as good as optimal
  - sometimes does **worse than FIFO**

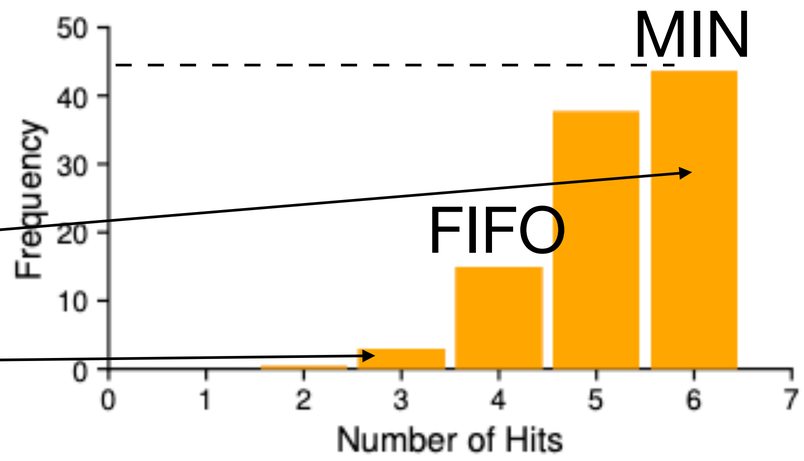



Figure 22.4: **Random Performance Over 10,000 Trials**

# Page replacement policy

- Problem: cache is small. Which page should be removed?
  - Option #1: **MIN** - evict the page that will not be used in the future
    - cons: need to predict the future
  - Option #2: **FIFO** - evict the oldest page
    - cons: more likely to throw out heavily used pages, Belady's anomaly
  - Option #3: **Random** - evict a page chosen at random
    - often used as a lower bound on the replacement policy

# LRU

- **Least recently used** - use history to guide selection
  - if a page was accessed **recently**, it is likely to be accessed **again**
  - evict the least recently used page
  - relies on **temporal locality**
    - e.g. if a process loops over the same structure frequently

# History-based policies

- LRU - **least recently** used
- LFU - **least frequently** used - evict pages that are not as often accessed
- The opposite is possible as well - ignore locality
  - MRU - **most recently** used - evict pages that are recently used
  - MFU - **most frequently** used - evict pages that are frequently used

# Page replacement policy

- Problem: cache is small. Which page should be removed?
  - Option #1: **MIN** - evict the page that will not be used in the future
    - cons: need to predict the future
  - Option #2: **FIFO** - evict the oldest page
    - cons: more likely to throw out heavily used pages
  - Option #3: **Random** - evict a page chosen at random
    - often used as a lower bound on the replacement policy
  - Option #4: **LRU** - evict the page that has not been used the longest
    - a good approximation of MIN

# Summary

- Virtual memory is **larger** than physical memory
- We still need to store information for virtual pages somewhere
- Swapping - since RAM is too small, use the **hard drive** to extend it
- RAM becomes a **cache** for virtual memory
  - **Page faults** occur because page is not in RAM
  - If RAM is full, need to replace pages

# Summary

- Replacement policies
  - **MIN** - remove pages that will not be used in the future - infeasible in real world
  - **FIFO** - remove oldest pages - simple but **ineffective** for most workloads
  - **RAND** - remove **random** pages - simple, but **inconsistent** by design
  - **LRU** - remove **least recently** used pages - use **history** to make eviction decisions

# Reading

- Chapter 21 Beyond Physical Memory: Mechanisms
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys.pdf
- Chapter 22 Beyond Physical Memory: Policies
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-beyondphys
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-beyondphys-policy
- No optional reading this week

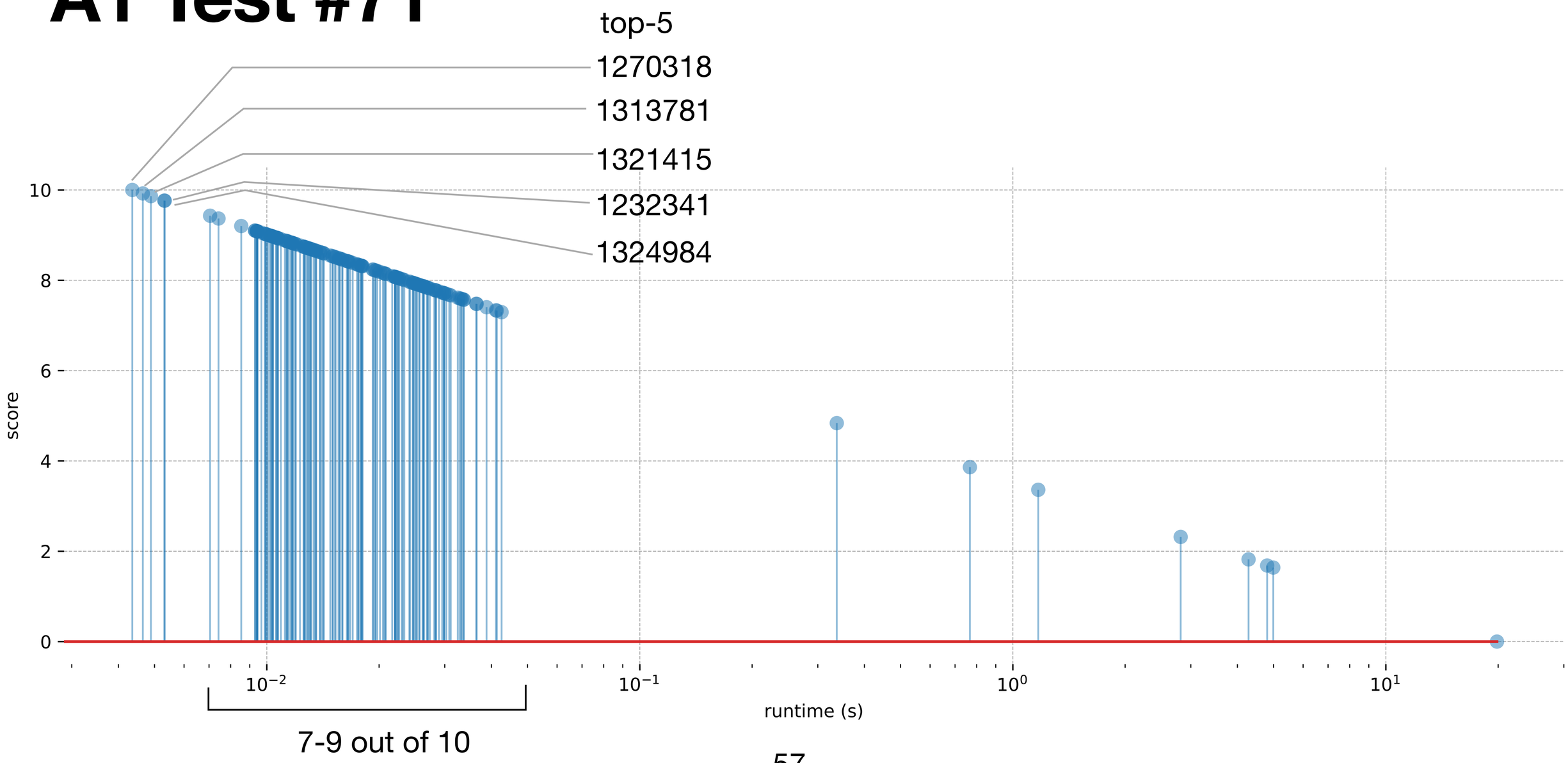
# A1 Test #71

- 163/251 (64%) submissions qualified for speed test ranking
  - 148 submissions passed 71 tests through autograder
  - 30 submissions passed 70 tests but failed test #71
    - 15 qualified after rerunning test #71
    - 15 did not qualify (runtime errors or >> 30 s runtime)
- used hyperfine* to test on a quiet system
  - hyperfine --runs 500 --warmup=3 "./rr input71.txt 10 1"
  - used total mean time (s) for rankings



*https://github.com/sharkdp/hyperfine

# A1 Test #71


top-5
1270318
1313781
1321415
1232341
1324984
score
10
8
6
4
2
0
$10^{-2}$
$10^{-1}$
$10^{0}$
$10^{1}$
runtime (s)
7-9 out of 10

# README

- Please **do not change** the header formatting
  - Just replace text with your name and student id (no caps)

README.md 1.08 KiB  Code  Preview

## CIS*3110 A2 Memory allocation

**Full Name: YOUR_NAME_GOES_HERE**

**Student id: STUDENT_ID_GOES_HERE**

### Overview

Briefly describe the project and explain the implementation of two memory allocation algorithms that are required by this assignment. Do not copy the assignment instructions, use your own words. You may provide pseudo-code if you wish. If you did some interesting optimization, you may include a description of that as well.

### Installation

Explain how to compile the code on Linux (optionally on other OSs).

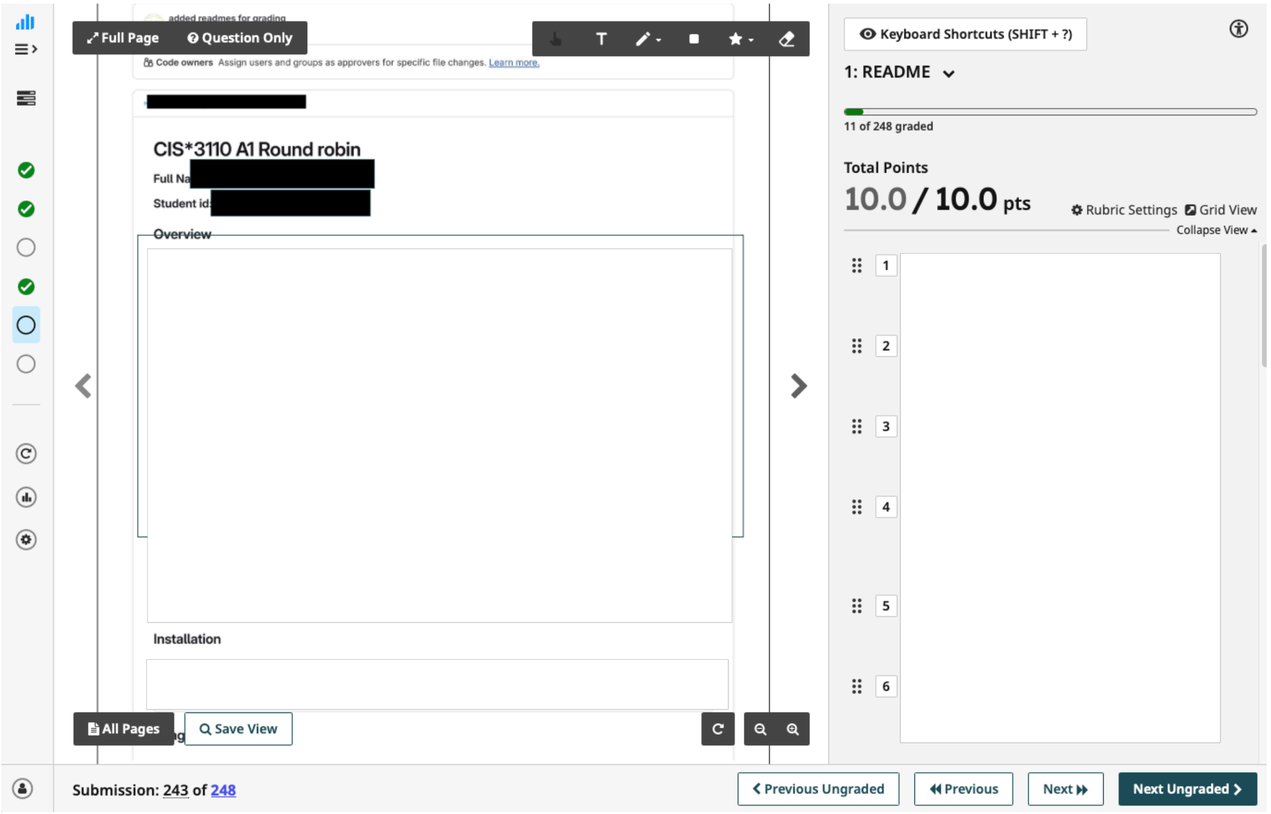
### Usage

Explain how to run the code, what are the parameters, inputs, pre-conditions, assumptions.

Explain the outputs and what they mean.

# README

- Please **do not change** the header formatting
  - Just replace the placeholders with your name and student id (no caps)
- Submissions are graded **anonymously** on Gradescope
- If your name is in the **wrong** place
  - your submission **will not be matched** to your name automatically
  - your submission **will not be anonymized** correctly



# README

- Still being graded, aim to release by the end of the week
- Common issues
  - **Language** (e.g. unclear, too casual)
  - **Completeness** (e.g. mention context switch but not explaining what it means)
  - **Formatting** (e.g. not using proper tags for code, not splitting text into paragraphs, bullet points)
    - Note: formatting is part of the A2 README criteria

# README

- What makes a good README
  - information should be **clearly presented**
    - **concise** description but with sufficient details
  - information should be **easy to find**
    - formatting to **highlight** important things
      - table of contents
      - sections/subsections, bullet points
      - tags for code
  - Installation and running instructions
  - Explanations of outputs

# README

dchangez_README.md

**added readmes for grading**
Yulia Kotseruba authored 2 hours ago

**Code owners** Assign users and groups as approvers for specific file changes. Learn more.

dchangez_README.md 6.38 KiB

## CIS*3110 A1 Round robin

**Full Name: Danial Changez**

[redacted]

### Table of Contents


- Overview
  - Algorithm Description
    - Pseudocode
  - Optimizations
    - File I/O
    - Manual Parsing
    - Circular Queue Array
    - Bitwise AND over Modulus
    - Process Batching
    - Process Jump
    - Compiler Arguments
- Installation
  - Compilation
- Usage
  - Running the Simulator
  - Input File Format
  - Output Format
  - Example Runs
- Test Generator


table of contents makes navigation easier

multiple ways of presenting information (text, pseudocode)

clear and concise writing

subsections, bullet points

### Overview

The project simulates a Round Robin (RR) CPU scheduling algorithm and computes the average turnaround, waiting, and response times across all processes.

### Algorithm Description

RR is a preemptive scheduling algorithm that assigns a fixed time slice (quantum) to each process in a queue:

1. Processes are loaded from a file in the `arrival,burst` format (one per line).
2. A queue holds processes that have arrived and waiting.
3. The scheduler picks the next process from the front of the queue.
4. The process executes for one quantum (or till completion if remaining time < quantum).
5. If the process did not finish within the quantum, it is placed at the rear of the queue.
6. Context switch overhead is added between process switches unless:
   - Process first run (computed after context switch).
   - Idle CPU.
   - No immediate execution of another process after its predecessor.
   - When a single process is in the queue.
7. This continues until all processes complete.

#### Pseudocode

```
while processes > 0:
    Add processes where arrival <= clock to queue
    if queue is not empty:
        process = dequeue()
        execute for min(quantum, remaining_time)

        if process not finished:
            enqueue(process)
        else:
            record metrics in struct

        if process switches and criteria met:
          clock += context_switch
    else:
      clock += quantum
```

### Optimizations

Here are some optimizations made to reduce simulation time. Please note, some may not be effective due to modern compiler optimizations, but were tested before this was realized.

#### File I/O

- Utilized the `sys/stat.h` library to determine size needed to load inputs into memory.
- Directly imported the entire file into memory within one operation through `fread()` to avoid multiple system calls.

#### Manual Parsing

- Manually parsed the inputs into an array to avoid system calls.
- Array stores all `Process` structs, and any index can be accessed in O(1) time.

#### Circular Queue Array

- A linear queue would require shifting **n** elements backwards if the queue was full, costing **O(n)** time.
- A circular queue combats this by wrapping the rear element to the first index once it reaches the end of the array.

#### Bitwise AND over Modulus

- Modulus uses floating point arithmetic, costing hundreds of cycles to compute.
- Bitwise AND can be completed in one CPU cycle, cutting down total computation significantly with the constant dequeue and enqueue operations of RR.
  - **Note:** Modern C compilers may optimize this automatically for modulus operations of power 2, so the benefit depends on the test environment.

#### Process Batching

- When there is a single process in queue with `burst > quantum`, the calculation can be completed immediately by skipping the partitioning of quanta and immediately moving to the next process.
- **Note**: This only holds true in simulations, not a real environment.

#### Process Jump

- If the CPU is idle, there is no need to increment the clock by enough quanta until arrival of the next process.

# README

- Directly jump to the next process instead.

**Compiler Arguments**

- **O3**: An aggressive flag that adds several optimizations (Refer to official Documentation for a comprehensize list).
- **FLTO**: An optimization that automatically inlines all functions.
  - Must be specifed at both compile and link time.
  - Merges all object files into a single internal image.
  - The merge allows the compiler to inline functions from different files.

## Installation

### Compilation

**Linux/MacOS/Windows (with MinGW):**

- The make file has been setup to detect your OS, and adjust the make commands to match.
- Run the following command to generate an executable from the project.

```
make clean; make
```

## Usage

### Running the Simulator

- The executable **requires** the input file path, with quantum and context_switch as optional.
- By default, `quantum = 10` and `context_switch = 0`

```
./rr <input_file> [quantum] [context_switch]
```

- If the test directory is missing, run (will work on Linux/MacOS/Windows):

```
$ mkdir tests
```

### Input File Format

Each line contains a process in the format `arrival,burst`:

```
0,10
5,20
10,15
```

### Output Format

The program outputs three space-separated values:

```
<avg_turnaround> <avg_waiting> <avg_response>
```

### Example Runs

- Lets create a custom test file first.
- The command to run the executable is the same on all systems.

Linux/MacOS:

```
$ echo -e "0,24\n4,3\n5,3" > tests/default.txt
```

Windows:

```
> Write-Output "0,24`n4,3`n5,3" > tests/default.txt
```

**Example 1: Running with Defaults**

```
./rr tests/default.txt
16.67 6.67 4.67
```

**Example 2: Quantum of 4**

```
./rr tests/default.txt 4
14.00 4.00 2.00
```

**Example 3: Quantum of 4 and Context Switch of 1**

```
./rr tests/default.txt 4 1
16.67 6.67 2.67
```

### Test Generator

- A Python script (`src/test_generator.py`) is included to generate random test input files for testing and demos.
- You must have Python3 installed to run this (installed on Linux by default on most distros).
- For optional script parameters, run the script with the `--help` tag.

```
python3 ./src/test_generator.py [optional_args]
```

**Example: Generate default test file**

```
python3 ./src/test_generator.py
Generated 100000 processes to tests/test_input.txt...

./rr tests/test_input.txt 10 1
36479584.16 36479085.00 546091.55
```

extra: explanation of compiler arguments and link to resources

clear installation instructions for major OSs*

extra: universal Makefile

clear usage instructions for major OSs

clear input and output formatting (uses proper md tags for readability)

extra: script for generating inputs

* if you don't have access to all major OSs, you can just say that it was tested on a particular system. The point here is clarity. If you don't say where this code can run, some users will waste their time trying something untested or known not to work.

# README

- This example is **not the only way** to do it
- Study this and other examples linked in the A2 instructions
- **Develop your own style**

# Quiz 9

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999